INTRODUCTION TO MACHINE LEARNING (LPCIT-114)

LAB PRACTICAL FILE

GURU NANAK DEV ENGINEERING COLLLEGE, LUDHIANA

BACHELOR OF TECHNOLOGY

(INFORMATION TECHNOLOGY)

SUBMITTED TO

PF. RUPINDER KAUR

SUBMITTED BY

NAME: ALISHA

UNIVERSITY ROLL NO:2203792

BRANCH:IT

SECTION-A

S. No	NAME OF THE PRACTICAL	DATE OF PRACTICAL	TEACHER SIGNATURE
1.	Python libraries		
2.	Implement Simple Linear Regression.		
3.	Implement Random Forest Regression		
4.	Implement Logistic Regression		
5.	Implement Decision Tree classification algorithms		
6.	Implement k-nearest neighbours classification algorithm		
7.	Implement Naive Bayes classification algorithms		
8.	Implement K-means clustering to Find Natural Patterns in Data		
9.	Implement K- Mode Clustering.		
10.	Evaluating Machine Learning algorithm with balanced and unbalanced datasets		
11.	Compare various Machine Learning algorithms based on various performance metrics		

PRACTICAL NO.1
PYTHON LIBRARIES

1.Numpy

NumPy is a Python library used for working with arrays. It also has functions for working in domain of linear algebra, Fourier transform, and matrices. NumPy was created in 2005 by Travis Oliphant. It is an open-source project and you can use it freely. NumPy stands for Numerical Python.

Code:

```
import numpy as np

x=np.array([1,2,3])

y=np.array([[1,2],[3,4]])

z=np.array([[[1,2],[3,4],[5,6]]])

print(x)

print(y)

print(z)

a1=np.zeros((3,3))

a2=np.ones((2,2))

a3=np.arange(0,10,2)

print(a1)

print(a2)

print(a3)

a1=np.array([1,2,3,4])

print("element =",a1[2])

print("element =",a1[-1])

print("Range of Elements:",a1[0:3])
```

Output:

```
[1 2 3]
[[1 2]
 [3 4]]
[[[1 2]
  [3 4]
  [5 6]]]
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
[[1. 1.]
 [1. 1.]]
[0 2 4 6 8]
element = 3
element = 4
Range of Elements: [1 2 3]
```

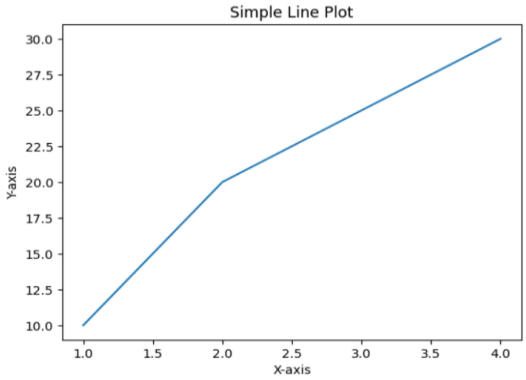
2. **Matplotlib**

Matplotlib is a low-level graph plotting library in python that serves as a visualization utility. Matplotlib was created by John D. Hunter. Matplotlib is open source and we can use it freely. Matplotlib is mostly written in python, a few segments are written in C, Objective-C and JavaScript for Platform compatibility.

Code:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]

y = [10, 20, 25, 30]

plt.plot(x, y)

plt.xlabel('X-axis')

plt.ylabel('Y-axis')

plt.title('Simple Line Plot')

plt.show()
```

Output:



2.Pandas

A powerful data manipulation library used for data analysis and manipulation. It provides two main data structures: Series and DataFrame.

Data cleaning, manipulation, data frames, time series analysis.

Code:

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35]}

df = pd.DataFrame(data)

print(df)
```

Output:

```
      Name  Age
0    Alice   25
1      Bob   30
2  Charlie   35
```

3.SciPy

Built on top of Numpy, SciPy is used for scientific and technical computing. It provides many algorithms for optimization, integration, interpolation, eigenvalue problems, and more.

Code:

```
from scipy import integrate

result = integrate.quad(lambda x: x**2, 0, 1)

print(result)
```

Output:

```
(0.3333333333333337, 3.700743415417189e-15)
```

4.PyTorch

A deep learning framework primarily used for neural networks and tensor computations. It provides dynamic computation graphs, unlike TensorFlow, which uses static graphs.

Deep learning, neural networks, optimization.

Code:

```
import torch

# Example: Create a tensor

x = torch.tensor([1.0, 2.0, 3.0])
```

print(x)

Output:

```
tensor([1., 2., 3.])
```

5.Scikit-learn

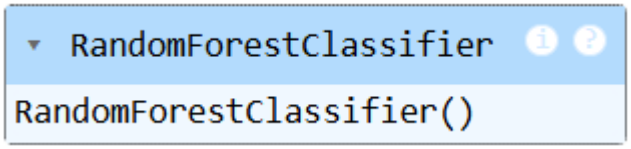
A powerful library for machine learning in Python. It supports various supervised and unsupervised algorithms and is built on top of NumPy and SciPy.

Classification, regression, clustering, dimensionality reduction, and other machine learning tasks.

Code:

```
from sklearn.ensemble import RandomForestClassifier

# Example: Train a RandomForest model

clf = RandomForestClassifier()

X = [[1, 2], [3, 4], [5, 6]]

y = [0, 1, 0]

clf.fit(X, y)
```

Output:



6. Seaborn

Seaborn is a data visualization library which is based on Matplotlib. It is very helpful in creating beautiful statistical plots with minimal code.

Key Features:

Seaborn has many high-level functions that simplify the process of creating complex statistical visualization.

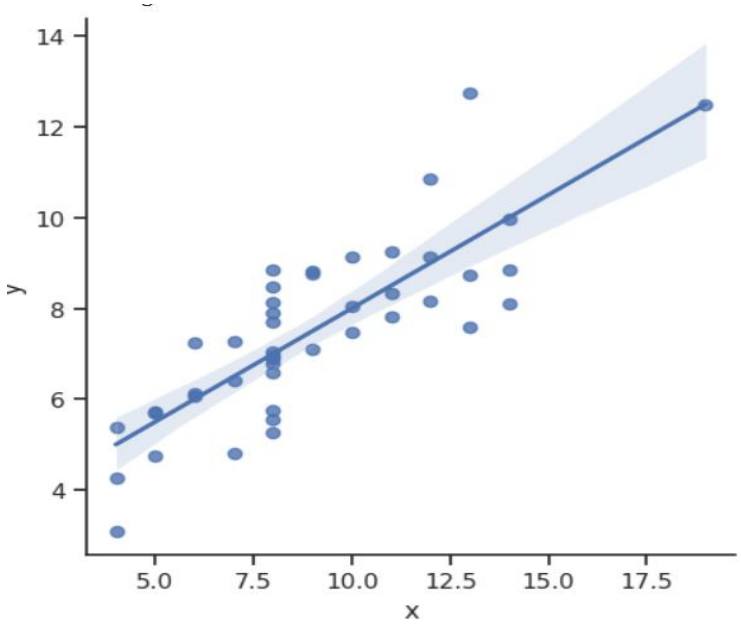
The themes and color palettes built-in chances the visual appeal of the plots.

It works perfectly with Pandas DataFrames, it takes DataFrames as input making it easier for users working with tabular data.

Code:

```
import seaborn as sns
sns.set(style="ticks")
df = sns.load_dataset("anscombe")
sns.lmplot(x="x", y="y", data=df)
```

Output:



PRACTICAL NO.2

SIMPLE LINEAR REGRESSION

Simple Linear Regression is a statistical method used to model the relationship between two variables:

- **One independent variable (X)** — the predictor
- **One dependent variable (Y)** — the response

The goal is to find a **linear equation** that best predicts the value of **Y** based on **X**. The equation is generally written as:

The general form of the simple linear regression equation is:

$Y = \beta_0 + \beta_1 X + \varepsilon$

Where:

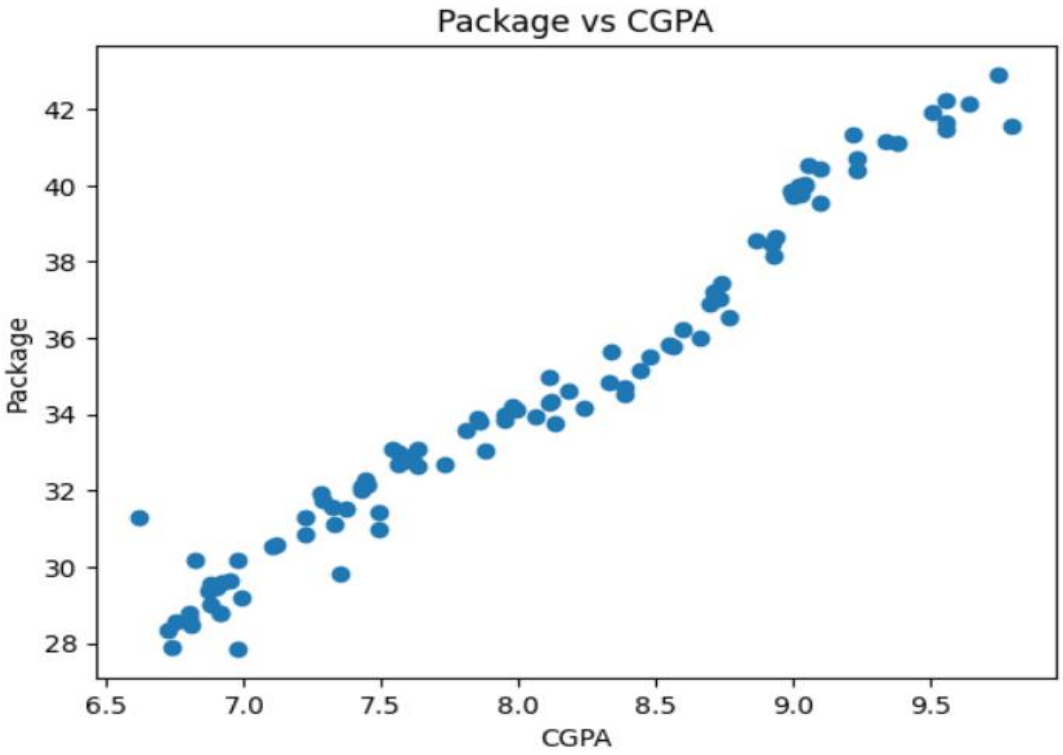
- Y is the dependent variable.
- X is the independent variable.
- β_0 is the y-intercept (the value of Y when X is 0).
- β_1 is the slope (the change in Y for each unit change in X).
- ε is the error term (the difference between the actual value of Y and the predicted value).

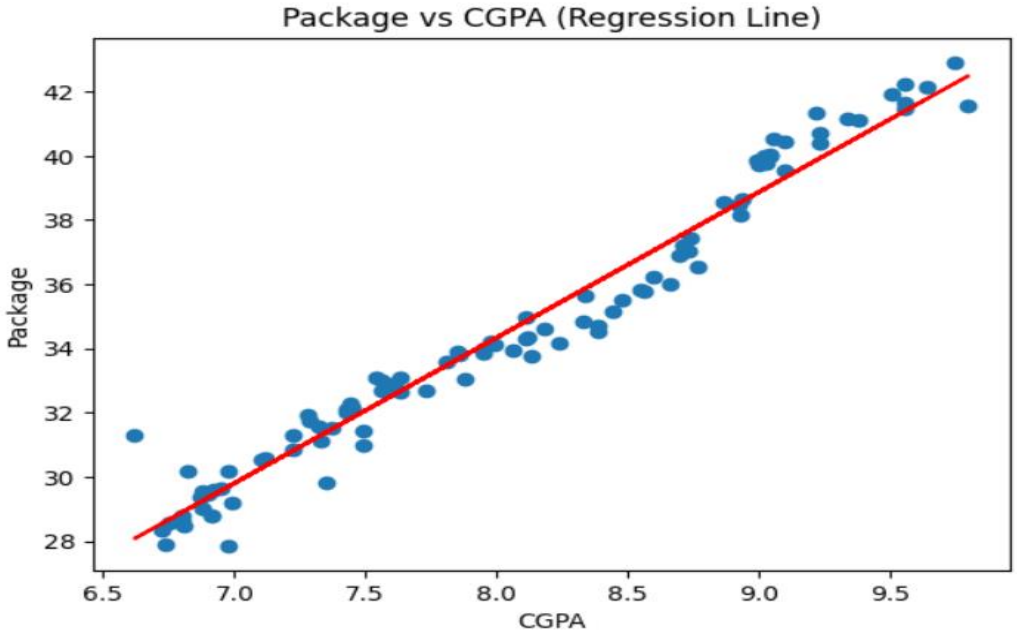
Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error

plt.scatter(df['CGPA'], df['Package'])

plt.xlabel('CGPA')

plt.ylabel('Package')

plt.title("Package vs CGPA")

plt.show()


X = df.iloc[:, 0:1]  # Features (CGPA)
```

```
y = df.iloc[:, -1]   # Labels (Package)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=2)
lr = LinearRegression()

# Train the model using the training data
lr.fit(X_train, y_train)

# Predict the package for the test set
y_pred = lr.predict(X_test)

# Visualize the linear regression line
plt.scatter(df['CGPA'], df['Package'])
plt.plot(X_train, lr.predict(X_train), color="red")
plt.xlabel('CGPA')
plt.ylabel('Package')
plt.title("Package vs CGPA (Regression Line)")
plt.show()
# Output the slope and intercept of the line
print(f"Slope (m): {lr.coef_[0]:.2f}")
print(f"Intercept (b): {lr.intercept_:.2f}")
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
print("\nError Metrics:")
print(f"Mean Absolute Error (MAE): {mae:.2f} LPA")
print(f"Mean Squared Error (MSE): {mse:.2f} LPA²")
print(f"Root Mean Squared Error (RMSE): {mse**0.5:.2f} LPA")
```

OUTPUT:






```
Slope (m): 4.53
Intercept (b): -1.92
```

MULTI LINEAR REGRESSION

Multiple Linear Regression (MLR) is an extension of simple linear regression that models the relationship between **one dependent variable (Y)** and **two or more independent variables (X_1, X_2, ..., X_n)**.

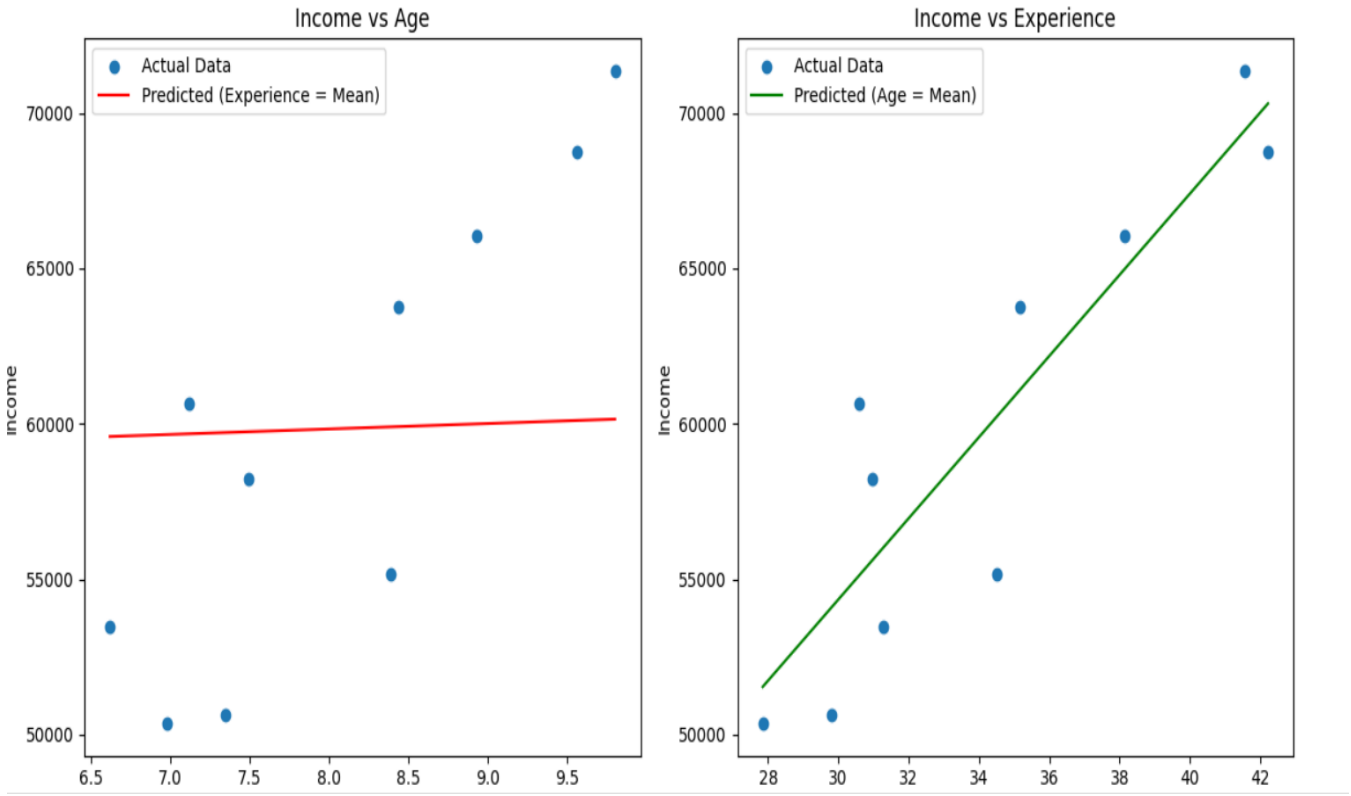
It is used when the outcome (Y) is believed to be influenced by **several predictors**, and it helps estimate the effect of each one while holding the others constant.

CODE:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression


# Data (X1: age, X2: experience)

X1 = np.array([7.49, 9.80, 8.93, 8.39, 6.62, 7.35, 6.98, 9.56, 8.44, 7.12])

X2 = np.array([30.97, 41.56, 38.15, 34.51, 31.29, 29.81, 27.85, 42.22, 35.16, 30.59])

income = 1000 * X1 + 1500 * X2 + np.random.normal(0, 5000, len(X1))  # Simulating income


# Create DataFrame

df = pd.DataFrame({'age': X1, 'experience': X2, 'income': income})


# Prepare the data for the model

X = df[['age', 'experience']]

y = df['income']


# Fit the model

model = LinearRegression().fit(X, y)


# 2D Plot for Age vs Income (holding experience constant)

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)

exp_mean = df['experience'].mean()
```

```
age_range = np.linspace(df['age'].min(), df['age'].max(), 100)
pred_income_age = model.predict(pd.DataFrame({'age': age_range, 'experience': [exp_mean]*100}))
plt.scatter(df['age'], y, label='Actual Data')
plt.plot(age_range, pred_income_age, 'r-', label='Predicted (Experience = Mean)')
plt.xlabel('Age')
plt.ylabel('Income')
plt.title('Income vs Age')
plt.legend()

# 2D Plot for Experience vs Income (holding age constant)
plt.subplot(1, 2, 2)
age_mean = df['age'].mean()
exp_range = np.linspace(df['experience'].min(), df['experience'].max(), 100)
pred_income_exp = model.predict(pd.DataFrame({'age': [age_mean]*100, 'experience': exp_range}))
plt.scatter(df['experience'], y, label='Actual Data')
plt.plot(exp_range, pred_income_exp, 'g-', label='Predicted (Age = Mean)')
plt.xlabel('Experience')
plt.ylabel('Income')
plt.title('Income vs Experience')
plt.legend()

plt.tight_layout()
plt.show()
```

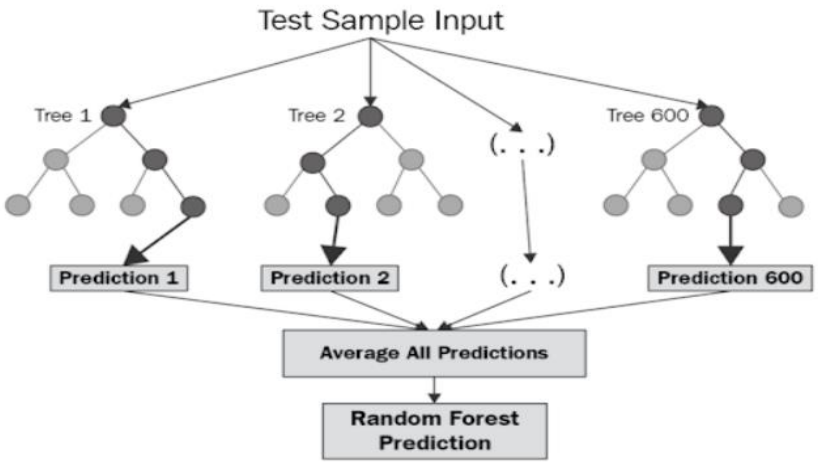
OUTPUT:


Income vs Age
Actual Data
Predicted (Experience = Mean)
income
70000
65000
60000
55000
50000
6.5
7.0
7.5
8.0
8.5
9.0
9.5
Income vs Experience
Actual Data
Predicted (Age = Mean)
Income
70000
65000
60000
55000
50000
28
30
32
34
36
38
40
42

PRACTICAL NO.3

RANDOM FOREST REGRESSION

A random forest is an ensemble learning method that combines the predictions from multiple decision trees to produce a more accurate and stable prediction. It is a type of supervised learning algorithm that can be used for both classification and regression tasks.



Code:

```
from google.colab import files

uploaded = files.upload()

import pandas as pd

df = pd.read_csv('taxi_trip_pricing.csv')

df.head()

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestRegressor

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import OneHotEncoder

from sklearn.compose import ColumnTransformer

from sklearn.pipeline import Pipeline

from sklearn.metrics import mean_squared_error, r2_score
```

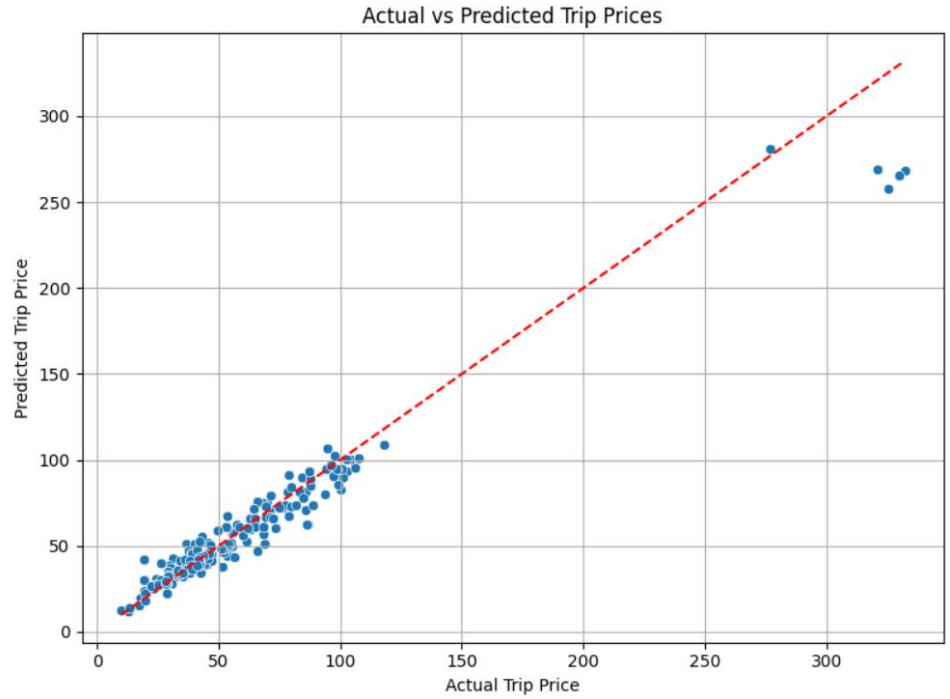
```
# Load dataset
df = pd.read_csv("taxi_trip_pricing.csv")
# Drop rows with missing target values
df = df.dropna(subset=["Trip_Price"])
# Define features and target
X = df.drop("Trip_Price", axis=1)
y = df["Trip_Price"]

# Identify numerical and categorical columns
num_cols = X.select_dtypes(include="number").columns.tolist()
cat_cols = X.select_dtypes(include="object").columns.tolist()

# Preprocessing pipelines
numeric_transformer = SimpleImputer(strategy="mean")
categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, num_cols),
    ("cat", categorical_transformer, cat_cols)
])

# Create model pipeline
model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", RandomForestRegressor(n_estimators=100, random_state=42))
])
```

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Train model
model.fit(X_train, y_train)
# Predict and evaluate
y_pred = model.predict(X_test)
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R² Score:", r2_score(y_test, y_pred))

# Plot: Actual vs Predicted
plt.figure(figsize=(8, 6))
sns.scatterplot(x=y_test, y=y_pred)
plt.xlabel("Actual Trip Price")
plt.ylabel("Predicted Trip Price")
plt.title("Actual vs Predicted Trip Prices")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', linestyle='--')
plt.grid(True)
plt.tight_layout()
plt.show()
# Plot: Feature Importance
# Extract trained RandomForestRegressor
regressor = model.named_steps["regressor"]

# Get feature names from preprocessing pipeline
encoded_feature_names = (
    model.named_steps["preprocessor"]
    .transformers_[0][2] +  # numeric
    list(model.named_steps["preprocessor"]
         .transformers_[1][1]
         .named_steps["encoder"]
```

```
        .get_feature_names_out(cat_cols))
)
importances = regressor.feature_importances_
feature_imp_df = pd.DataFrame({"Feature": encoded_feature_names, "Importance": importances})
feature_imp_df = feature_imp_df.sort_values(by="Importance", ascending=False)
# Plot top 15 features
plt.figure(figsize=(10, 6))
sns.barplot(data=feature_imp_df.head(15), x="Importance", y="Feature", palette="viridis")
plt.title("Top 15 Feature Importances")
plt.xlabel("Importance Score")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```

Output:

```
Mean Squared Error: 128.58661725433973
R² Score: 0.9449868313049891
```



PRACTICAL NO.4

Implement Logistic Regression.

When data scientists may come across a new classification problem, the first algorithm that may come across their mind is Logistic Regression. It is a supervised learning classification algorithm which is used to predict observations to a discrete set of classes. Practically, it is used to classify observations into different categories. Hence, its output is discrete in nature. Logistic Regression is also called Logit Regression.

Logistic Regression Equation

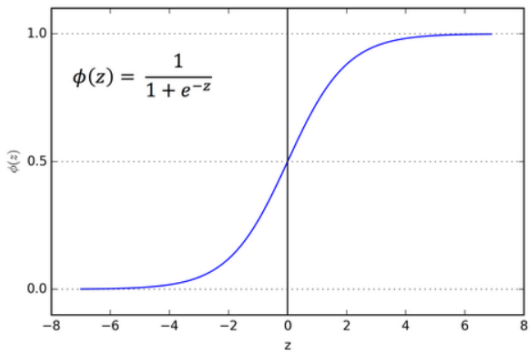
$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n)}}$$

Where:

- $\mathbf{x} = (x_1, x_2, \ldots, x_n)$ are the features
- β_0 is the intercept (bias term)
- $\beta_1, \beta_2, \ldots, \beta_n$ are the coefficients (weights)
- The entire expression is passed through the **sigmoid function** to map output to a probability between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{where } z = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n$$

Sigmoid Function

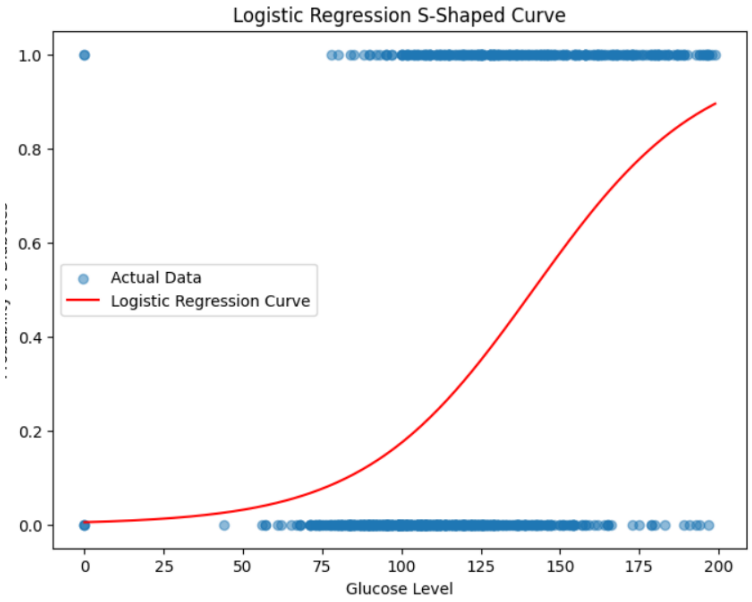


Code:

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.linear_model import LogisticRegression

from sklearn.model_selection import train_test_split

# Load dataset
```

```
df = pd.read_csv("diabetes2.csv")


X = df[["Glucose"]]
y = df["Outcome"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LogisticRegression()
model.fit(X_train, y_train)
X_range = np.linspace(X.min(), X.max(), 300).reshape(-1, 1)
y_prob = model.predict_proba(X_range)[:, 1]
# Plot results
plt.figure(figsize=(8, 6))
plt.scatter(X, y, label="Actual Data", alpha=0.5)
plt.plot(X_range, y_prob, color="red", label="Logistic Regression Curve")
plt.xlabel("Glucose Level")
plt.ylabel("Probability of Diabetes")
plt.title("Logistic Regression S-Shaped Curve")
plt.legend()
plt.show()
```

Output:

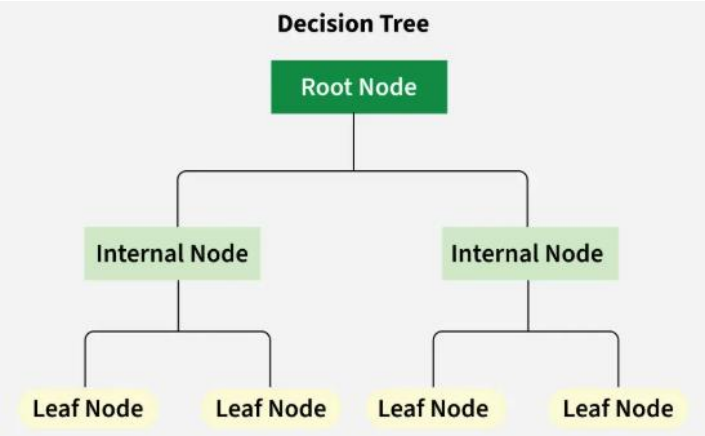


PRACTICAL NO.5

Implement Decision Tree classification algorithms

A decision tree is a graphical representation of different options for solving a problem and show how different factors are related.

- **Root Node** is the starting point that represents the entire dataset.
- **Branches**: These are the lines that connect nodes. It shows the flow from one decision to another.
- **Internal Nodes** are Points where decisions are made based on the input features.
- **Leaf Nodes**: These are the terminal nodes at the end of branches that represent final outcomes or predictions

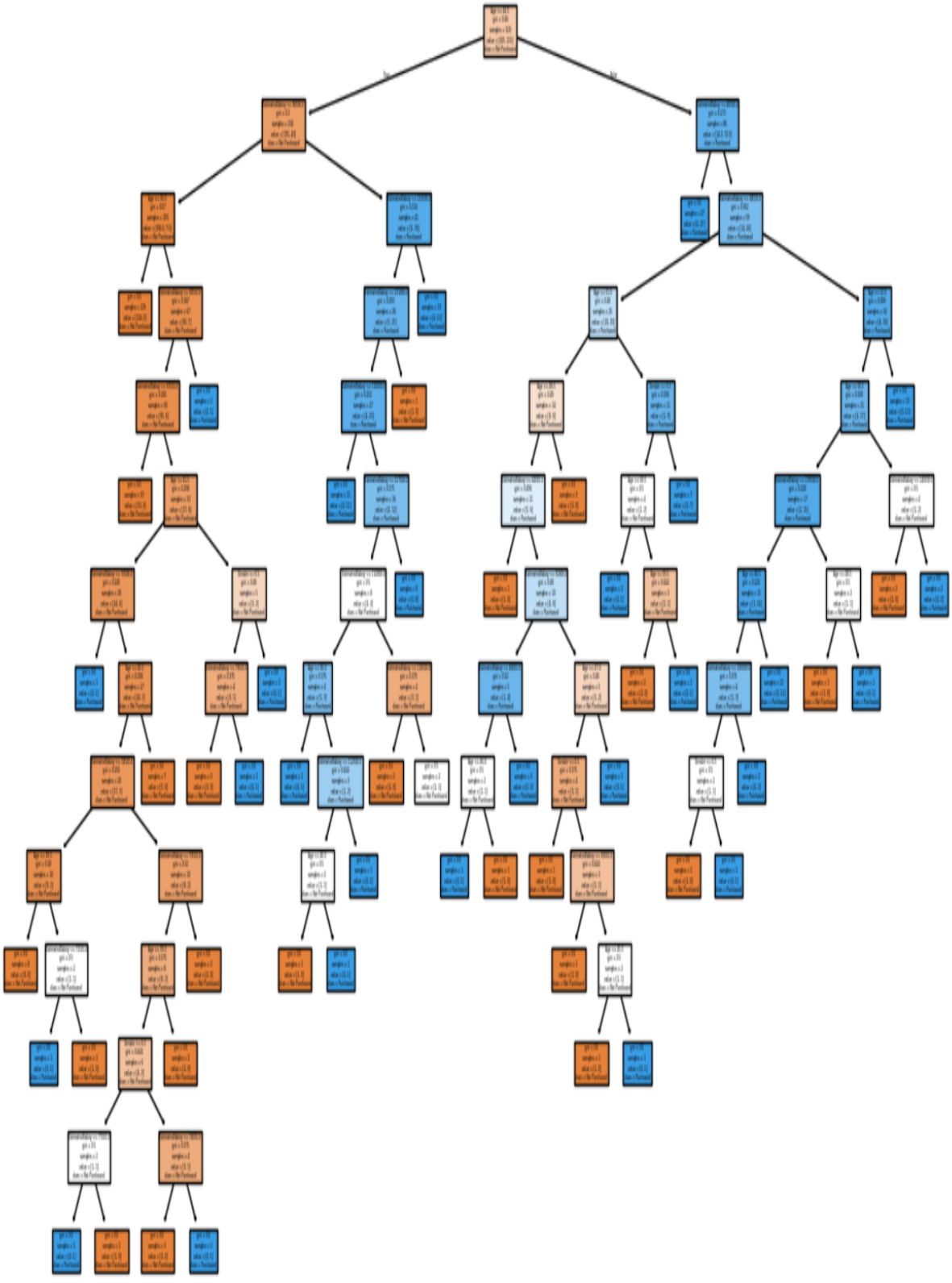


CODE:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder

from sklearn.tree import DecisionTreeClassifier, plot_tree


# Load dataset

df = pd.read_csv("Social_Network_Ads.csv")


df = df.drop(columns=["User ID"])


# Encode categorical variables
```

```
label_encoder = LabelEncoder()
df["Gender"] = label_encoder.fit_transform(df["Gender"])

# Split data into features and target
X = df.drop(columns=["Purchased"])
y = df["Purchased"]

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train a Decision Tree model
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Visualize the Decision Tree
plt.figure(figsize=(12, 8))
plot_tree(model, feature_names=X.columns, class_names=["Not Purchased", "Purchased"],
filled=True)
plt.show()
```

Output:



PRACTICAL NO. 6

Implement k-nearest neighbours classification algorithms.

The K-Nearest Neighbors (K-NN) algorithm is a simple, instance-based (or lazy) machine learning method used for classification and regression. It makes predictions based on the majority class of its nearest neighbors in the feature space.

Euclidean distance:

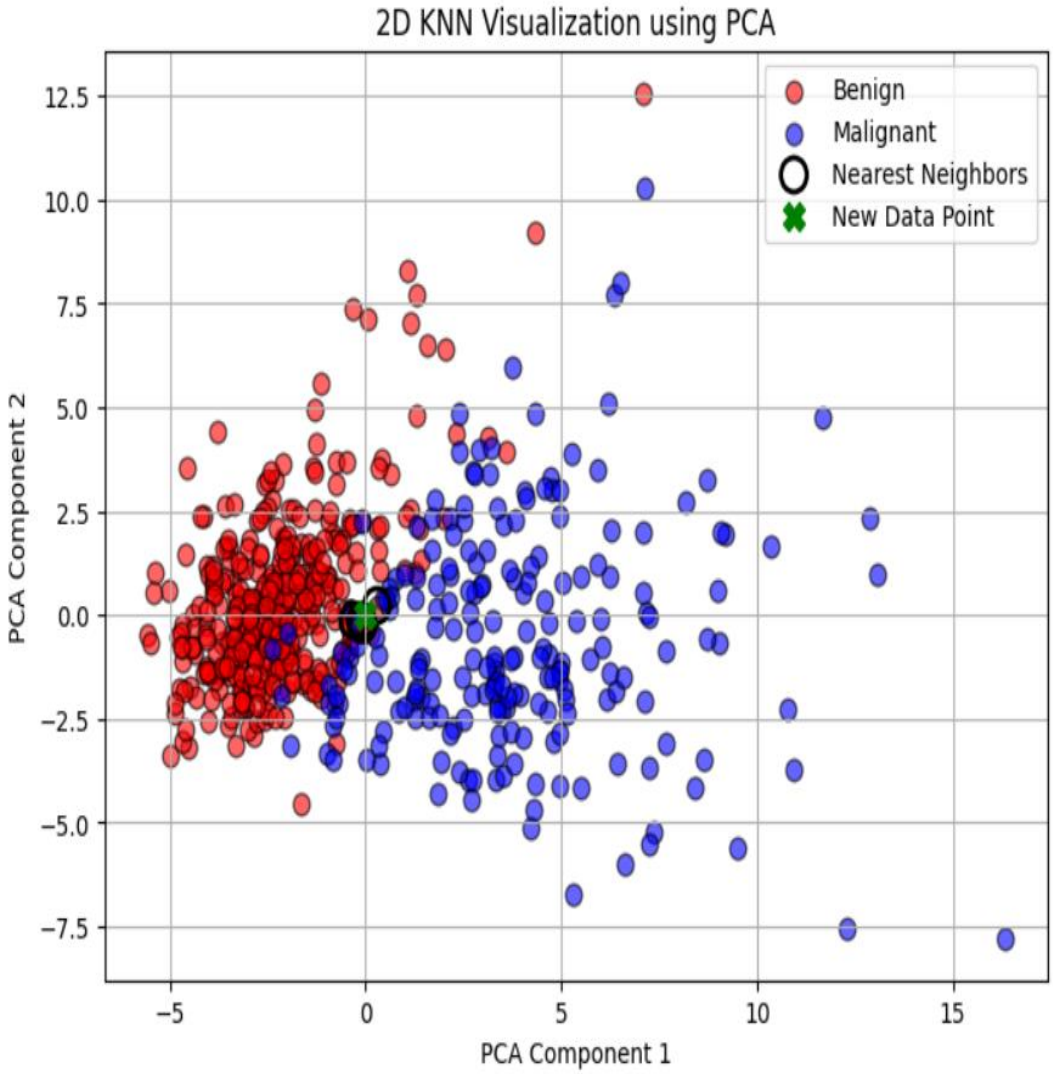
$$d(x, x_i) = \sqrt{(x_1 - x_{i1})^2 + (x_2 - x_{i2})^2 + \cdots + (x_n - x_{in})^2}$$

CODE:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.decomposition import PCA

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.model_selection import train_test_split

# Load and clean data

df = pd.read_csv("KNNAlgorithmDataset.csv")

df.drop(['id', 'Unnamed: 32'], axis=1, inplace=True)

df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

# Features and target

X = df.drop('diagnosis', axis=1)

y = df['diagnosis']

# Split the dataset

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# Reduce to 2D with PCA for visualization

pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X_scaled)

# Train KNN on PCA-reduced data
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_pca, y)

# Create a synthetic "new data point" for illustration
new_point = np.array([[0, 0]])  # In PCA space
predicted_class = knn.predict(new_point)[0]
neighbors = knn.kneighbors(new_point, return_distance=False)

# Plot
plt.figure(figsize=(8, 6))
for label, color, name in zip([0, 1], ['red', 'blue'], ['Benign', 'Malignant']):
    plt.scatter(X_pca[y == label, 0], X_pca[y == label, 1],
                c=color, label=name, edgecolor='k', alpha=0.6, s=60)

# Highlight neighbors
plt.scatter(X_pca[neighbors[0], 0], X_pca[neighbors[0], 1],
            facecolors='none', edgecolors='black', s=150, linewidths=2, label='Nearest
Neighbors')
# Plot the new point
plt.scatter(new_point[0, 0], new_point[0, 1], c='green', s=100, label='New Data Point',
marker='X')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.title('2D KNN Visualization using PCA')
plt.legend()
plt.grid(True)
plt.show()
```

Output:


2D KNN Visualization using PCA
Benign
Malignant
Nearest Neighbors
New Data Point
PCA Component 2
PCA Component 1
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−5
0
5
10
15

PRACTICAL NO 7:

Implement Naive Bayes classification algorithms

Naive Bayes is a probabilistic classifier based on Bayes' Theorem, with the naive assumption that features are independent given the class label.

Bayes' Theorem

$$P(C \mid X) = \frac{P(X \mid C) \cdot P(C)}{P(X)}$$

Where:

- CCC: Class label (e.g., "spam" or "not spam")
- XXX: Feature vector (e.g., words in an email)
- P(C|X)P(C|X)P(C|X): Posterior probability of class given features
- P(X|C)P(X|C)P(X|C): Likelihood (probability of features given the class)
- P(C)P(C)P(C): Prior probability of the class
- P(X)P(X)P(X): Evidence (can be ignored in classification since it's the same for all classes)

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report


# Step 1: Load the dataset
df = pd.read_csv('drug200.csv')


# Step 2: Encode categorical variables
le = LabelEncoder()
for column in df.columns:
    if df[column].dtype == 'object':
        df[column] = le.fit_transform(df[column])
```

```
# Step 3: Features and target
X = df.drop('Drug', axis=1)
y = df['Drug']

# Step 4: Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 5: Train Naive Bayes classifier
model = GaussianNB()
model.fit(X_train, y_train)

# Step 6: Predict and evaluate
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

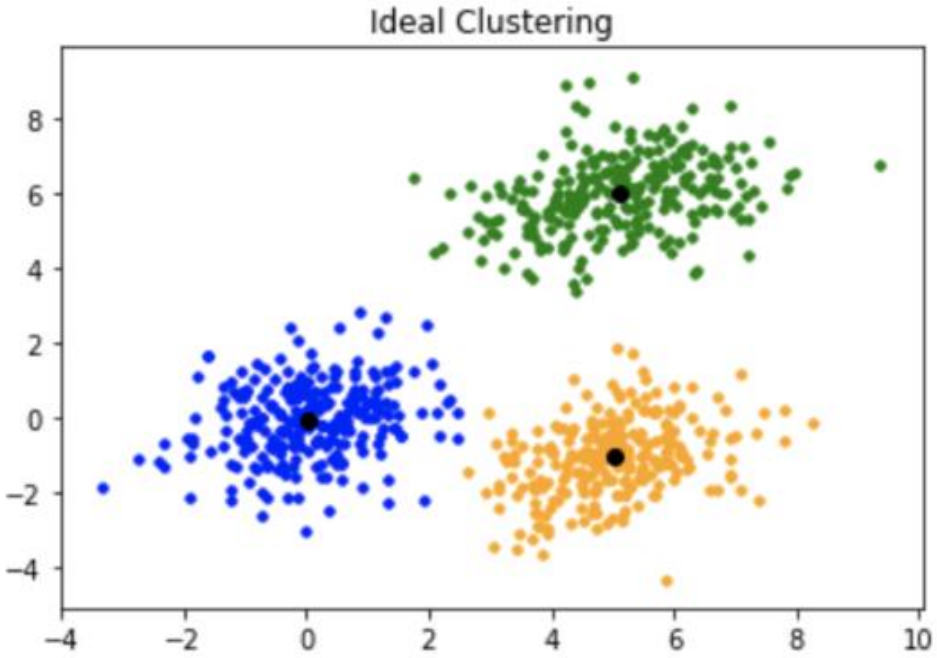
Output:

```
Accuracy: 0.925

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.80      0.89        15
           1       0.86      1.00      0.92         6
           2       0.75      1.00      0.86         3
           3       0.83      1.00      0.91         5
           4       1.00      1.00      1.00        11

    accuracy                           0.93        40
   macro avg       0.89      0.96      0.92        40
weighted avg       0.94      0.93      0.92        40
```

PRACTICAL NO..8

Implement K-means clustering to Find Natural Patterns in Data

K-Means Clustering is an unsupervised machine learning algorithm used to find natural groupings or patterns in data by partitioning it into K clusters. It's ideal when you don't have labeled data and want to explore hidden structures.



CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
# Load the dataset
data = pd.read_csv("cluster_data.csv")
# Apply K-means clustering
num_clusters = 3  # You can adjust this based on your data distribution
kmeans = KMeans(n_clusters=num_clusters, random_state=42, n_init=10)
data["Cluster"] = kmeans.fit_predict(data)
# Plot the clusters
plt.figure(figsize=(8, 6))
```

```
for cluster in range(num_clusters):
    cluster_data = data[data["Cluster"] == cluster]
    plt.scatter(cluster_data["Feature 1"], cluster_data["Feature 2"], label=f"Cluster {cluster}")
# Plot cluster centroids
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], c="red", marker="X", s=200, label="Centroids")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("K-Means Clustering")
plt.legend()
plt.show()
```

Output: